Question -1 – Decrease and Conquer

You are given a weighted directed graph G = (V, E) with non-negative edge weights and a source vertex s. Design a decrease-and-conquer algorithm (Dijkstra's Algorithm) to find the shortest path from s to all other vertices.

Tasks

1. Describe how the problem size decreases at each step of Dijkstra's algorithm.
2. Implement the algorithm using a min-priority queue.
3. Trace the algorithm on a sample graph with 6 vertices and show the evolution of distance estimates at each step.
4. Derive and analyze the time complexity for both an adjacency matrix and a binary heap implementation.
5. Experimentally compare both implementations for dense and sparse graphs.

Question – 2 Dynamic Programming

You are given a rod of length n and a price table p[1..n] where p[i] is the price of a rod of length i. Design a dynamic programming algorithm to determine the maximum revenue obtainable by cutting the rod into pieces and selling them.

Tasks

1. Identify the optimal substructure and derive the recurrence relation.
2. Implement the bottom-up dynamic programming solution.
3. Modify the algorithm to also output the actual cuts that yield maximum revenue.
4. Analyze the time and space complexity of the algorithm.
5. Extend the problem: add a fixed cutting cost c per cut and redesign the recurrence accordingly.

Question – 3 – Divide and Conquer

You are given an array A[0...n−1] containing both positive and negative integers representing daily profit or loss.

Design a divide-and-conquer algorithm to determine the maximum sum of a contiguous subarray.

Your algorithm should consider three possibilities:

1. Maximum subarray lies entirely in the left half
2. Maximum subarray lies entirely in the right half
3. Maximum subarray crosses the midpoint

Tasks

1. Design the divide-and-conquer algorithm.
2. Implement the algorithm.
3. Derive the recurrence relation for the running time.
4. Solve the recurrence using the Master Theorem.
5. Compare the performance with the brute-force $O(n^2)$ solution.